

ELASTICO

FORENSIC AUDIT REPORT

Complete System, Product, UX, UI, and Security Audit

Prepared by: Principal Product Engineer, UX Architect, Senior Frontend Engineer,
Backend Auditor, Data Visualization Specialist, and QA Lead

August 2026

CONFIDENTIAL

Table of Contents

- 1. Executive Summary**
- 2. Product Health Score**
- 3. What Works (Verified Functional)**
- 4. What Does Not Work (Verified Failures)**
- 5. What Is Partially Implemented**
- 6. What Is Fake or Mocked**
- 7. UX Problems**
- 8. UI Problems**
- 9. Information Architecture Problems**
- 10. Analytics and Data Visualization Problems**
- 11. AI System Problems**
- 12. Technical Architecture Problems**
- 13. Security Problems**
- 14. Performance Problems**
- 15. Top 10 Fixes**
- 16. What Should NOT Be Changed**
- 17. Implementation Roadmap**

1. Executive Summary

ELASTICO is an AI-powered football analytics platform built on Next.js, deployed on Vercel, backed by a PostgreSQL database via Prisma ORM. The application aims to serve football analysts, scouts, and serious bettors with predictions, live scores, tactical analysis, and AI-powered insights. This audit was conducted by examining every source file, every API route, every component, and every data source in the repository. No assumptions were made. Every claim in this report is backed by specific file paths, line numbers, and observed code behavior.

The application presents a polished, dark-themed interface with glassmorphism design, neon accents, and extensive use of Recharts for data visualization. However, beneath the surface, the product has a significant gap between what the UI promises and what the backend actually delivers. The core data pipeline (ESPN live scores, StatsBomb event data, Prisma database operations, authentication) is genuine and functional. The prediction engine uses real stochastic mathematics (Merton Jump-Diffusion, GARCH, Monte Carlo simulation). The AI chat uses a legitimate 7-provider failover gateway.

The critical finding is that **six of the twenty-two Elastico components contain fabricated data presented as real**. The admin dashboard is 100% theater, with 16 random-number generators producing fake metrics. The tactical analysis view shows static fake data for any match selected. The social feed is entirely invented. The player view uses hardcoded radar stats and random form charts. The system monitor fakes its integrity score with Math.random() regardless of actual API responses. The compare view shows identical head-to-head data for every team pair. These are not fallback states; they are the primary data sources for these views.

On the UX and UI front, the application suffers from several structural problems: a single-page architecture that loads all 21 views eagerly with no code splitting, zero loading states on the primary dashboard, 19 navigation items in the sidebar creating information overload, duplicate data displayed across multiple dashboard cards, non-functional zoom controls, dead UI elements (forgot password, terms links), and approximately 200 lines of unused CSS. The design system, while visually sophisticated, contains inconsistencies in border radius values, duplicate color definitions, and no reduced-motion support.

The backend is architecturally sound with 42 API route files, 16 external API integrations, and legitimate mathematical models. However, security concerns exist around the demo login endpoint (which can auto-create admin accounts), in-memory rate limiting that resets on server restart, and security utility functions that are defined but never imported by most routes. The authentication system uses proper JWT with bcrypt hashing and account lockout, which is a genuine strength.

2. Product Health Score

Each dimension is scored out of 10 based on verified evidence, not aspirations. Scores reflect the current deployed state of the application, not its potential.

Dimension	Score	Rationale
Functionality	4.5	Core data pipeline works. Six views are fake. Multiple dead UI elements.
UX	3.0	No loading states, 19 nav items, duplicate data, no URL routing, dead buttons.

Dimension	Score	Rationale
UI	5.5	Visually polished but ~200 lines dead CSS, inconsistent icons, no light theme, no reduced-motion.
Data Integrity	3.5	49 mock data sources across 8 files. Admin/tactical/social 100% fabricated.
Analytics Quality	4.0	Real math engines but tactical/compare charts show fake data. Dashboard has no loading states.
AI Quality	6.0	Legitimate 7-provider gateway. Mock fallback is transparent in metadata. No conversation history.
Performance	3.5	All 21 views eagerly imported. No code splitting. React Query installed but unused.
Accessibility	4.0	Good ARIA labels on nav. Missing aria-current, no skip links, no reduced-motion, dead keyboard targets.
Security	5.0	Proper JWT/bcrypt/lockout. Demo login creates admins. In-memory rate limits. Security utils unused.
Production Readiness	2.5	Fake data in 6 views. No input validation on most routes. No CI. PWA disabled. Prisma schema issues.

Overall: 4.2 / 10 - The application has a strong mathematical foundation and legitimate data pipeline, but is undermined by pervasive fabricated data in multiple views, missing loading states, dead UI elements, and architectural decisions that prevent it from functioning as a credible production application.

3. What Works (Verified Functional)

The following features have been verified through code inspection as genuinely functional, connecting UI to real data through working API endpoints.

3.1 Authentication System

The authentication system is fully implemented and production-grade. Login, registration, and session management use JWT tokens signed with HS256, bcryptjs password hashing with 10 salt rounds, and a 7-day token expiry. Account lockout is enforced after 5 failed login attempts with a 30-minute cooldown. Rate limiting is applied at 10 login attempts and 5 registration attempts per minute per IP address. Session restoration on page load validates the stored token against the `/api/auth/me` endpoint and properly cleans up `localStorage` on validation failure. The system properly strips sensitive fields (`passwordHash`, `twoFactorSecret`, `sessionId`) from API responses.

Evidence: `src/lib/auth.ts` (JWT, bcrypt, lockout), `src/app/api/auth/login` (rate limiting), `src/app/api/auth/register`

3.2 ESPN Live Data Pipeline

The ESPN public API integration is the backbone of ELASTICO and it works without any API key. The `/api/live` endpoint supports 12 distinct action types (scores, standings, news, leaders, injuries, odds, probability, play-by-play, leagues, date-specific) across 20 football leagues. The `/api/news` endpoint uses a three-tier priority chain: ESPN direct fetch first, then Newsdata.io if available, then database fallback. The `/api/sync` endpoint fetches live ESPN scores and upserts them into the database. This data flows into the dashboard live score ticker, the matches view, and the news feed.

Evidence: `src/app/api/live/route.ts` (12 actions, 20 leagues), `src/app/api/news/route.ts` (3-tier chain)

3.3 Prediction Engine (Mathematical Core)

The prediction engine is the strongest technical asset of ELASTICO. It implements four genuine mathematical models: ELO rating system, Poisson regression, Dixon-Coles model, and Stochastic Monte Carlo simulation with 150,000 configurable runs. The engine in `src/lib/prediction-engine.ts` (807 lines) implements Merton Jump-Diffusion processes, GARCH(1,1) volatility calibration, Bivariate Correlated Poisson modeling, and Kelly Criterion capital allocation. The `/api/predictions/compute` endpoint fetches live ESPN scores and computes a 4-model ensemble producing Asian handicap lines, over/under 2.5, BTTS probabilities, most likely scorelines, and confidence scores. The `/api/advanced-analytics` endpoint implements 20 proprietary formulas across five analytical tiers. This is real mathematics, not placeholder logic.

Evidence: `src/lib/prediction-engine.ts` (807 lines), `src/app/api/predictions/compute/route.ts`

3.4 AI Gateway

The AI chat uses a legitimate 7-provider failover chain: Google Gemini, Groq, Cerebras, Mistral, NVIDIA, GitHub Models, and OpenRouter. Each provider has a cooldown system (429 errors trigger 120s cooldown, 403/401 trigger 300s, timeouts trigger 30s). Streaming is supported via Server-Sent Events with `TransformStream`. The `/api/chat` endpoint gathers real match context from the database (teams,

stats, players, predictions, community votes) before sending to the AI provider. When no AI keys are configured, it falls back to a deterministic template based on ELO ratings, which is marked as mock-fallback in the response metadata. The fallback is honest about what it is.

Evidence: `src/lib/ai-gateway.ts` (7 providers, cooldown, streaming), `src/app/api/chat/route.ts`

3.5 Database Operations

All core CRUD operations (matches, teams, players, predictions, votes, bookmarks, notifications, achievements, admin) use genuine Prisma database queries against PostgreSQL via Neon. The schema has 15 models with proper foreign key relationships. The admin user management, announcements, feature flags, settings, and audit logs are all real database operations. The leaderboard endpoint correctly aggregates prediction accuracy and golden boot statistics with minimum prediction count thresholds.

Evidence: `prisma/schema.prisma` (15 models), `src/app/api/matches/`, `/api/admin/`, `/api/leaderboard/`

3.6 Match Detail and Predictions

The match detail view fetches real data from `/api/matches/[id]`, including team statistics, lineups, match events (goals, cards, substitutions), xG data, vote distributions, and prediction breakdowns. Users can submit predictions via `/api/predictions` with confidence levels and model selection. Community voting works via `/api/matches/[id]` with proper upsert patterns and activity logging. Match simulation via `/api/matches/[id]/simulate` generates random events based on team ELO ratings, updates ELO values, and evaluates existing predictions.

Evidence: `src/components/elastico/match-detail-view.tsx`, `src/app/api/matches/[id]/route.ts`

4. What Does Not Work (Verified Failures)

#	Severity	Component	Issue	Evidence
1	P0	Zoom Controls	Header zoom buttons (+/-/reset) change store value but it is never applied to any DOM element	page.tsx:151 reads zoomLevel; no CSS transform or font-size references it
2	P0	Forgot Password	Button exists with no onClick handler - completely dead UI element	login-view.tsx:181 - no onClick bound
3	P0	Terms/Privacy Links	Links point to href="#" - broken navigation to nonexistent pages	login-view.tsx:272-274
4	P0	Admin Dashboard	All 16 data generators use Math.random() - 100% fabricated metrics displayed as real	admin-view.tsx:155-363 (generateUserGrowth through generateRateLimits)
5	P1	Tactical Analysis	All 16 data constants are static regardless of match selection - same data for every match	tactical-view.tsx:72-178 (PRESSING_HEATMAP through BUILD_UP_PATTERNS)
6	P1	Social Feed	Entire view uses hardcoded MOCK_USERS, MOCK_FEED, DISCUSSION_THREADS - zero API calls	social-view.tsx:42-163 (all 6 constants are primary data)
7	P1	Compare View H2H	Head-to-head data is identical for every team pair selected	compare-view.tsx:59-68 - comment says "Head-to-head mock data"
8	P1	System Monitor Integrity	Integrity score always uses Math.random() (88-97) regardless of API response	system-monitor-view.tsx:352 - score generated before API result is used
9	P1	Dashboard Loading	Zero loading states across 683 lines - blank dashboard on slow connections	dashboard-view.tsx - no Skeleton, spinner, or loading indicator found
10	P1	Header Search Bar	handleSearchFocus callback is defined but never called - dead code	header.tsx:102-104 (dead function), line 17 (unused Input import)
11	P2	7 Missing View Titles	Players, Compare, Achievements, Export, Social, Prediction Engine, System Monitor all show "ELASTICO" in header	header.tsx:36-52 - viewTitleMap missing 7 entries
12	P2	Bookmark Toggle (Matches)	Clicking bookmark only changes local state - never calls /api/bookmarks	matches-view.tsx:147 - setBookmarked(!bookmarked) with no API call
13	P2	Market Analysis Card	Dashboard card is permanently empty placeholder - no data path exists	dashboard-view.tsx:478-498 - always shows "Select a match to view"
14	P2	Prediction Models Card	Dashboard card is permanently empty placeholder	dashboard-view.tsx:500-521 - always shows "Prediction models require a selected match"

5. What Is Partially Implemented

#	Severity	Component	Issue	Evidence
1	P1	Player View	MOCK_PLAYERS used as fallback but FORM_CHART_DATA, RADAR_STATS, generateRadarStats are all primary fake data	player-view.tsx:75-121
2	P1	AI Chat	Real 7-provider gateway exists, but falls back to hardcoded template when no keys configured. No conversation history.	chat-view.tsx, src/app/api/chat/route.ts
3	P1	Achievements	MOCK_ACHIEVEMENTS is fallback (API exists), but CHALLENGES and MOCK_LEADERBOARD tabs have no API	achievements-view.tsx:60-99
4	P1	Export View	Real export API exists (/api/export), but MOCK_EXPORT_HISTORY is primary and post-export rows/size are fabricated	export-view.tsx:62-68, lines 135,138
5	P2	System Monitor	Real API calls to /api/system/* exist, but fallback branches fake success with [Mock] prefix	system-monitor-view.tsx:256-277, 307-314, 322-328
6	P2	Prediction Engine TimesFM	Real NVIDIA NIM call exists, but falls back to weighted moving average mock on failure	src/app/api/prediction-engine/timesfm/route.ts
7	P2	PWA	Service worker registration is explicitly disabled ("cache fix"). Old SW is auto-unregistered via script tag.	use-pwa.ts (disabled), layout.tsx:56 (unregister script)
8	P2	Subscription	Static pricing page with 3 plans. No payment integration. No Stripe/billing backend.	subscription-view.tsx - entire file is static HTML
9	P3	Theme System	next-themes installed and ThemeProvider configured, but only dark mode has CSS. enableSystem=false, html hardcoded dark.	layout.tsx:50, globals.css:122-154 (duplicate dark block)

6. What Is Fake or Mocked

This section catalogs every instance of fabricated data in the frontend. Each entry is classified by whether the mock data serves as the primary data source (the user always sees fake data) or a fallback (the user sees fake data only when the API fails). A total of 49 distinct mock/fake data sources were identified across 8 component files.

6.1 Primary Fabricated Data (No API Path Exists)

The following components present fabricated data as real with no corresponding API endpoint to replace them. Users will always see fake data when visiting these views, regardless of configuration.

#	Severity	Component	Issue	Evidence
1	P0	Admin Dashboard	16 generate*() functions + 1 realtime updater = 17 fake data sources. User growth, API usage, revenue, activity feed, model performance, heatmap, funnel, feature usage, A/B tests, subscriptions, news items, realtime metrics, geo data, content calendar, audit trail, rate limits. All use Math.random() or hardcoded values.	admin-view.tsx:155-461
2	P0	Tactical View	16 static data constants (PRESSING_HEATMAP, XG_TIMELINE, SHOT_MAP, PASS_NETWORK, MOMENTUM_DATA, ZONE_CONTROL, SET_PIECE_DATA, SUBSTITUTION_IMPACT, DEFENSIVE_ACTIONS, AERIAL_DUELS, WIDE_PLAY, COUNTER_ATTACK, BUILD_UP_PATTERNS, TRANSITION_SPEED). Same data for every match.	tactical-view.tsx:72-178
3	P0	Social View	6 constants (MOCK_USERS, MOCK_FEED, DISCUSSION_THREADS, TRENDING_TOPICS, COMMUNITY_STATS, inline leaderboard). Entire social layer is fabricated.	social-view.tsx:42-163, 475-479
4	P1	Compare View	h2h (5 hardcoded results, same for all pairs), eloHistory (Math.sin/cos formula), scoringTrends (6 identical arrays), 3 hardcoded stats in statComparisons, squadDepth (Math.random per render)	compare-view.tsx:59-137
5	P1	Player Radar Stats	RADAR_STATS hardcoded for 5 players. generateRadarStats() uses Math.random() for all others. FORM_CHART_DATA generates random 8-match form.	player-view.tsx:98-121
6	P2	Achievements Extras	CHALLENGES (6 fake daily/weekly challenges) and MOCK_LEADERBOARD (6 fake entries) have no API	achievements-view.tsx:83-99
7	P2	Export History	MOCK_EXPORT_HISTORY is primary. handleExport fabricates rows (10-60) and size (2-32KB) with Math.random()	export-view.tsx:62-68, 135, 138

6.2 Fallback Fabricated Data (API Exists, Mock on Failure)

These components attempt to fetch real data but fall back to fabricated data when the API fails. Users see fake data during outages or when API keys are not configured. The concern is that fallback data is not clearly labeled as degraded.

#	Severity	Component	Issue	Evidence
1	P1	System Monitor	runAudit falls back to fake HEALTHY status with [Mock] prefix. runIntegrityCheck always uses Math.random() score (88-97) ignoring API response. testInSandbox always returns success.	system-monitor-view.tsx:256-358
2	P2	Player List	MOCK_PLAYERS (20 hardcoded players) initializes state. /api/players replaces on success.	player-view.tsx:75-96, 128
3	P2	Achievements List	MOCK_ACHIEVEMENTS (19 fake definitions) initializes state. /api/achievements replaces on success.	achievements-view.tsx:60-81, 123
4	P2	AI Chat	When all 7 AI providers fail or have no keys, returns hardcoded template analysis based on ELO ratings. Metadata includes mock-fallback flag.	src/app/api/chat/route.ts
5	P2	TimesFM	Real NVIDIA NIM call, then fallback to nvidia/llama-3.1-70b, then double fallback to weighted moving average mock with trend slope analysis.	src/app/api/prediction-engine/timesfm/route.ts

7. UX Problems

UX problems are ranked by severity and user impact. Each problem was identified through systematic evaluation of user journeys, information hierarchy, cognitive load, navigation patterns, and interaction design.

#	Severity	Component	Issue	Evidence
1	P0	No Loading States	Dashboard (683 lines) has zero loading indicators. On slow connections, users see a blank page with empty cards. Same issue across most views - data fetches complete silently with no visual feedback during the wait.	dashboard-view.tsx, page.tsx:154-161 (5 parallel fetches with no UI feedback)
2	P0	Zero URL-Based Routing	Entire app is one page.tsx with 21 views switched via Zustand currentView state. No URL-based routing means: no browser back/forward navigation, no deep linking, no shareable URLs, no code splitting per route, no SSR/SSG per page. Users cannot bookmark specific views.	page.tsx:259-284 (switch statement), all 21 views eagerly imported lines 5-28
3	P1	19 Navigation Items	Sidebar has 19 items across 3 sections with no visual grouping differentiation. Analysis section has same font size and spacing as main nav. No section labels in collapsed mode. This exceeds the 7-plus-or-minus-two cognitive limit for navigation.	sidebar.tsx:55-92 (3 nav sections, 19 items)
4	P1	Duplicate Data Display	Dashboard shows the same ESPN match data three times: Live Scores card (lines 197-245), Latest Results, and All Matches card (lines 546-569). Three cards showing overlapping data from the same liveMatches array.	dashboard-view.tsx:197-245, 546-569
5	P1	Triple AI Chat CTA	Dashboard has 3 redundant paths to AI Chat: quick action button (line 636-641), AI Chat CTA card (lines 654-677), and sidebar navigation. The CTA card duplicates the quick action.	dashboard-view.tsx:636-677
6	P1	Empty Placeholders	Market Analysis and Prediction Models cards on dashboard are permanently empty with "Select a match" messages. They consume valuable above-fold space with zero value.	dashboard-view.tsx:478-521
7	P2	Dead Zoom Controls	Zoom buttons (+/-/reset) in header change Zustand store value but it is never applied to any DOM element. Feature is completely non-functional but occupies header space.	page.tsx:151 (read), header.tsx:240-267 (buttons)
8	P2	Duplicate Polling	page.tsx (line 228) and SetupView (line 78) both poll /api/setup every few seconds, creating redundant network requests during the database check phase.	page.tsx:228, SetupView:78

#	Severity	Component	Issue	Evidence
9	P2	No Empty States	Dashboard has no empty states for zero matches, zero teams, or zero news. If data arrays are empty (before API calls complete), cards render with nothing inside.	dashboard-view.tsx:349-360, 387-427
10	P2	Keyboard Shortcut Conflicts	Cmd+L not captured (browser uses for URL bar). Cmd+= and Cmd++ both try to zoom in. Cmd+K shows Mac symbol to all platforms including Windows/Linux.	page.tsx:172-189, sidebar.tsx:543, header.tsx:182
11	P3	Remember Me Decorative	Checkbox on login form is never sent to the API. It is purely decorative, creating a false expectation of persistent sessions.	login-view.tsx:47, 193

8. UI Problems

UI problems cover visual design, design system consistency, accessibility, and responsive behavior issues.

#	Severity	Component	Issue	Evidence
1	P1	~200 Lines Dead CSS	25+ unused CSS classes identified: glass-card-hover, glass-intel, glass-surface, neon-border, intel-border, bento-grid, bento-cell, nav-rail, ai-rail, intel-feed, ambient-dot, ambient-bar, confidence-*, form-w/d/l, command-bar, panel-highlight, data-stream keyframe, micro-hover keyframe, glow-line keyframe. This is approximately 25% of globals.css.	globals.css:283, 278, 211, 396-422, 526-573, 637-650, 664-711, 717, 753, 786
2	P1	No Reduced-Motion Support	18+ CSS animations run unconditionally. No @media (prefers-reduced-motion: reduce) query anywhere in the 794-line globals.css. Violates WCAG 2.3.3.	globals.css (entire file - no reduced-motion query)
3	P2	Duplicate .dark Block	Lines 122-154 (.dark {}) are a complete duplicate of lines 71-120 (:root {}). Since html has className="dark" hardcoded, only :root is needed. ~30 lines wasted.	globals.css:122-154
4	P2	Duplicate Color Definitions	--color-emerald and --color-neon-green both equal #00e676. --color-neon-blue and --color-intel are confusingly similar cyan shades.	globals.css:44,50,51,55
5	P2	Icon Collisions	Target icon used for both Predictions and Tactical. Shield icon used for both System Monitor and Admin Panel. Creates visual ambiguity in the sidebar.	sidebar.tsx:58 vs 68 (Target), 67 vs 91 (Shield)
6	P2	No aria-current on Nav	Active navigation items have no aria-current="page" attribute. Active state is purely visual, invisible to screen readers.	sidebar.tsx (all nav buttons)
7	P2	Password Toggle Inaccessible	Password visibility toggle buttons have tabIndex={-1}, making them invisible to keyboard navigation.	login-view.tsx:185
8	P2	Only Latin Font Subset	Geist Sans loaded with only "latin" subset. Football player names in Japanese, Arabic, or Korean will use fallback fonts with potentially inconsistent rendering.	layout.tsx:8,13
9	P3	Scroll Below Touch Target	Custom scrollbar is 5px wide, below the 9px minimum recommended for touch targets.	globals.css:175-188
10	P3	Duplicated getInitials()	getInitials() function is duplicated in sidebar.tsx:555 and header.tsx:112. Should be a shared utility.	sidebar.tsx:555, header.tsx:112

9. Information Architecture Problems

The information architecture of ELASTICO has fundamental structural issues that affect discoverability, comprehension, and user efficiency.

9.1 Single-Page Architecture

The decision to implement all 21 views as client-side state switches within a single page.tsx has significant consequences. The URL never changes from "/", which means users cannot bookmark specific views, share links to specific data (like a match or player), use browser navigation history, or benefit from Next.js code splitting. All 21 component files are eagerly imported at the top of page.tsx (lines 5-28), which means the entire application JavaScript is loaded on first visit regardless of which view the user needs. The installed @tanstack/react-query library is completely unused - the app uses manual fetch() calls inside Zustand actions instead of a proper data fetching library with caching, deduplication, and background refetching.

9.2 Navigation Overload

The sidebar presents 19 navigation items in three sections: Main (7 items), Analysis (8 items), and Bottom (4 items). The analysis section items (Prediction Engine, System Monitor, Tactical, Players, Compare, Achievements, Export, Social) have no visual distinction from the main navigation items - same font size, same spacing, same icon style. In collapsed mode, users see a wall of 19 icons with no grouping context. Research consistently shows that navigation should not exceed 7-9 items without grouping or progressive disclosure. The current structure creates cognitive overload, especially for new users who must scan 19 items to find what they need.

9.3 Feature Classification Gaps

The sidebar mixes core features (Dashboard, Matches, Predictions, AI Chat) with niche features (System Monitor, Export, Social, Achievements) at the same hierarchy level. Features like Tactical, Players, and Compare are analytical tools that could be grouped under an "Analysis" expandable section. The Admin Panel is a power-user feature that should be accessible but not prominent. The Subscription page is a meta-feature (about the product itself, not about football) that does not belong in the same navigation tier as analytical tools.

10. Analytics and Data Visualization Problems

#	Severity	Component	Issue	Evidence
1	P0	Tactical Charts Are Static	All 16 tactical visualizations (radar charts, heatmaps, shot maps, passing networks, xT timelines, momentum curves, zone control) display the same static data regardless of which match is selected. They are decorative, not analytical.	tactical-view.tsx:72-178
2	P0	Admin Charts Are Random	All charts in the admin dashboard (AreaChart, BarChart, LineChart, PieChart) display data generated by Math.random(). Every page load shows different fake trends, user growth curves, and revenue data.	admin-view.tsx:155-321

#	Severity	Component	Issue	Evidence
3	P1	Compare Charts Show Identical Data	ELO history uses Math.sin/cos formula producing the same wave pattern. Scoring trends are 6 identical arrays. H2H results are 5 hardcoded matches. The comparison is meaningless.	compare-view.tsx:59-107
4	P1	Player Radar Charts Use Random Stats	Only 5 players have hardcoded radar stats. All other players get Math.random()-generated stats. The radar visualization is misleading for most players.	player-view.tsx:103-121
5	P2	Dashboard Chart Misnamed	Bar chart labeled "Total Goals per Match" but the variable is named xgChartData (expected goals). The label and data source are inconsistent.	dashboard-view.tsx:94, 366-384
6	P2	Accuracy Ring Uses Approximate Pi	SVG circle strokeDasharray uses 3.14 instead of Math.PI * 100. Works by coincidence at 100% but the formula is imprecise.	dashboard-view.tsx:449
7	P2	Array Mutation in Sort	teams.sort() called directly on store array (line 408) mutates original data, which can cause subtle re-render bugs across components sharing the store.	dashboard-view.tsx:408, 532
8	P3	No Chart Empty States	Charts have no empty/loading/error states. If data is empty, Recharts renders an empty container with no user guidance.	All components using Recharts

11. AI System Problems

The AI system is one of the stronger parts of ELASTICO, but it has meaningful gaps that affect the user experience and the credibility of AI-generated insights.

11.1 No Conversation History

The `/api/chat` endpoint is completely stateless. Each request is processed independently with no memory of previous messages in the conversation. The chat view in the frontend maintains messages in Zustand state, but these are never sent back to the API as context. This means the AI cannot reference earlier questions, build on previous analysis, or maintain a coherent multi-turn conversation. For a product positioned as an AI analytics assistant, this is a significant limitation. Users expect conversational continuity from any modern AI chat interface.

Evidence: `src/app/api/chat/route.ts` - no conversation ID, no message history in request processing

11.2 Model Selector Is Cosmetic

The chat view offers three model options: "ELASTICO Pro (Best Quality)", "ELASTICO Fast (Low Latency)", and "ELASTICO Local (Offline Mode)". However, the `/api/chat` API endpoint does not accept a model parameter. All three options map to the same AI gateway failover chain. The user perception of choosing between quality and speed is fabricated. The "Local" option is particularly misleading as there is no local model - it triggers the same remote API calls.

Evidence: `chat-view.tsx:35-39 (MODEL_OPTIONS)`, `src/app/api/chat/route.ts` - no model parameter in request body)

11.3 No Source Attribution

When the AI provides analysis, there is no indication of which data sources informed its response. The system prompt includes match context (teams, stats, predictions), but the user cannot verify which data points the AI used or distinguish between grounded analysis and model hallucination. For a product targeting football professionals who make decisions based on data, the lack of source attribution undermines trust in AI outputs.

11.4 Mock Fallback Not Visually Distinguished

When all AI providers are unavailable and the system falls back to the hardcoded template, the response metadata includes a mock-fallback flag, but the frontend does not display any visual indicator. The user receives a deterministic template response (based on ELO ratings and team stats) that looks identical to a real AI response. This is a trust violation - the user believes they are receiving AI-generated analysis when they are reading a template.

Evidence: `chat-view.tsx` - no check for mock-fallback metadata in message rendering

12. Technical Architecture Problems

12.1 Installed but Unused Dependencies

The package.json includes several libraries that are installed but never meaningfully used in the application. @tanstack/react-query (v5.82.0) is a sophisticated data fetching library that is completely bypassed in favor of manual fetch() calls in Zustand actions. @tanstack/react-table (v8.21.3) is installed but not directly imported in any component. @dnd-kit (v6.3.1 and v10.0.0) for drag-and-drop is installed but unused. embla-carousel-react (v8.6.0) is installed but unused. @mdxeditor/editor (v3.39.1) is installed but unused. react-markdown (v10.1.0) is installed but the chat view uses a custom markdown parser instead. react-resizable-panels (v3.0.3) is installed but unused. @reactuses/core (v6.0.5) is installed but unused. These unused dependencies increase bundle size and add maintenance burden without providing value.

12.2 No Unified League Configuration

League codes are defined independently in at least 4 separate locations: the ESPN live endpoint (20 leagues), API-Sports client (10 leagues), TheSportsDB client (10 leagues), football-data.org client (10 leagues with a code mapper). The matches-view.tsx defines its own LEAGUES constant with 20 entries. There is no single source of truth for league metadata. The prior session identified and fixed a critical code mismatch where football-data.org used PD/CL/EL/BL1/FL1/DED while the rest of the app used LIGA/UCL/UCL/BL/L1/ERE, causing silent lookup failures. This was patched with a resolveFdCode() mapper, but the fundamental fragmentation remains.

12.3 No Connection Pooling Configuration

The Prisma client in src/lib/db.ts detects Neon Postgres and sets connection pooling, but the pool size and timeout values are defaults. In a serverless Vercel environment, each function invocation may create a new connection. There is no explicit pool configuration, no connection timeout, and no connection retry logic documented. For a production application making 42+ API routes that all hit the database, this is a potential bottleneck during traffic spikes.

12.4 In-Memory State in Serverless

Several systems maintain state in memory: rate limiting (Map in rate-limit.ts), AI provider cooldowns (Map in ai-gateway.ts), and notification counters. In a serverless environment like Vercel, each function invocation may run in a separate isolate, meaning in-memory state is not shared across requests and resets on cold starts. The rate limiter will not actually limit requests across different serverless instances. The AI cooldown system may allow rapid retries from different instances. This is a fundamental architectural mismatch between the stateful patterns used and the stateless serverless deployment target.

13. Security Problems

#	Severity	Component	Issue	Evidence
1	P0	Demo Login Creates Admins	POST /api/auth/demo auto-creates users with specified role (including admin) if they do not exist. No password verification. An attacker who knows the endpoint can create admin accounts.	src/app/api/auth/demo/route.ts

#	Severity	Component	Issue	Evidence
2	P1	In-Memory Rate Limiting	Rate limiter uses in-memory Map. In Vercel serverless, each isolate has its own Map. Rate limits do not apply across instances. An attacker can bypass by hitting different instances.	src/lib/rate-limit.ts
3	P1	Security Utils Unused	src/lib/security.ts has sanitizeInput, CSRF token generation, email validation. Most API routes do not import these functions. Input sanitization and CSRF protection exist but are not enforced.	src/lib/security.ts (defined), most route files (not imported)
4	P1	No CSRF Protection	POST endpoints rely solely on Bearer token authentication. No CSRF tokens on mutation endpoints. Vulnerable to cross-site request forgery if token is stored in localStorage.	All POST/PATCH/DELETE route handlers
5	P2	JWT_SECRET Soft Check	Auth library warns but does not throw if JWT_SECRET is empty or too short. App starts with empty JWT_SECRET, then fails at actual auth usage with a cryptic error.	src/lib/auth.ts (import-time check)
6	P2	Setup Endpoint Power	POST /api/setup can create all tables and seed data. Protected by SETUP_TOKEN header, but if token is weak or leaked, an attacker can restructure the database.	src/app/api/setup/route.ts

14. Performance Problems

#	Severity	Component	Issue	Evidence
1	P0	No Code Splitting	All 21 views are eagerly imported in page.tsx:5-28. Entire application JavaScript loads on first visit. No React.lazy(), no dynamic imports. Estimated initial JS payload is very large.	page.tsx:5-28
2	P1	React Query Unused	@tanstack/react-query v5.82.0 is installed but the app uses manual fetch() in Zustand. No request deduplication, no caching, no background refetch, no optimistic updates.	package.json, src/store/use-elastico-store.ts
3	P1	5 Parallel Fetches on Auth	On session restoration, 5 API calls fire simultaneously: fetchMatches, fetchTeams, fetchNews, fetchNotifications, fetchLiveScores. No prioritization, no cancellation, no error handling per call.	page.tsx:154-161
4	P1	Array Sort Mutation	teams.sort() called directly on store array mutates shared state. Can trigger cascading re-renders across components that subscribe to the same store slice.	dashboard-view.tsx:408, 532
5	P2	Unused Dependencies	8+ installed libraries (@tanstack/react-table, @dnd-kit, embla-carousel-react, @mdxeditor/editor, react-markdown, react-resizable-panels, @reactuses/core) are never imported. They may be tree-shaken but still add to install time and bundle analysis.	package.json
6	P2	No Image Optimization	Team logos, player photos, and news images are rendered as raw img tags or background colors without Next.js Image optimization.	Various components

15. Top 10 Fixes

These are the 10 highest-impact fixes, ordered by dependency and user impact. Each fix addresses a root cause that may resolve multiple downstream issues.

[P0] 1. Remove or Label All Fake Data

The single most impactful change. Replace the 17 admin dashboard generators with real API calls to /api/admin/* endpoints. Replace the 16 tactical view constants with match-specific data from /api/matches/[id]/tactical. Replace the 6 social view constants with real API endpoints or remove the view entirely. For the compare view, fetch real H2H data or remove the H2H section. Until this is done, ELASTICO cannot be shown to any serious user because the product is fundamentally dishonest about what data is real.

Files: [admin-view.tsx](#), [tactical-view.tsx](#), [social-view.tsx](#), [compare-view.tsx](#)

[P0] 2. Add Loading States Everywhere

Add Skeleton components to the dashboard (already imported in matches-view but not dashboard). Every data-fetching view needs: initial skeleton, loading spinner for subsequent fetches, error state with retry, and empty state with guidance. The dashboard should show skeleton cards while the 5 parallel API calls complete. This is the minimum requirement for a production-quality user experience.

Files: `dashboard-view.tsx`, `all view components`

[P1] 3. Implement URL-Based Routing

Convert the Zustand view-switching architecture to Next.js App Router pages. Each view should be a separate `page.tsx` in `src/app/(dashboard)/[view]/` or similar. This enables: code splitting (only load the current view), URL-based navigation (browser back/forward, bookmarks, deep links), per-page metadata and SEO, and proper loading/error boundaries per route.

Files: `page.tsx`, `src/app/ structure`

[P0] 4. Restrict Demo Login Endpoint

The `/api/auth/demo` endpoint should not be able to create admin accounts. Either remove the auto-create behavior, restrict it to development environment only (checking `NODE_ENV`), or require the `SETUP_TOKEN` for demo account creation. In production, this endpoint should be disabled or rate-limited to a single use.

Files: `src/app/api/auth/demo/route.ts`

[P2] 5. Clean Up Dead Code and CSS

Remove ~200 lines of unused CSS from `globals.css`. Remove unused imports from `header.tsx` (Input, `handleSearchFocus`). Remove the non-functional zoom controls or implement them properly. Remove duplicate `.dark` block. Remove duplicate color definitions. Remove the 8+ unused npm dependencies from `package.json`. This cleanup reduces bundle size, improves maintainability, and removes misleading code.

Files: `globals.css`, `header.tsx`, `page.tsx`, `package.json`

[P1] 6. Restructure Navigation

Reduce sidebar to 7-9 items maximum. Group analytical tools (Tactical, Players, Compare, Prediction Engine) under an expandable "Analysis" section. Move meta features (Settings, Subscription, Admin) to a user menu dropdown. Add section labels in collapsed mode. This reduces cognitive load and makes the navigation scannable for new users.

Files: `sidebar.tsx`

[P1] 7. Add AI Conversation History

Implement server-side conversation storage (a Conversation model in Prisma) and send recent messages as context in each `/api/chat` request. Display a visual indicator when the AI falls back to mock-template mode. Make the model selector functional by passing the selected model tier to the API. These changes transform the AI chat from a stateless question-answering tool into a genuine analytical assistant.

Files: `src/app/api/chat/route.ts`, `chat-view.tsx`, `prisma/schema.prisma`

[P1] 8. Fix Dashboard Information Hierarchy

Remove the triple-displayed match data (keep one card, remove the other two). Remove the permanently empty Market Analysis and Prediction Models cards. Remove the redundant AI Chat CTA (keep only the sidebar entry and quick action). Reduce the right column from 9 cards to 4-5. Ensure the above-fold area answers: what is happening right now (live scores), what should I do next (next match prediction), and how am I performing (accuracy ring).

Files: `dashboard-view.tsx`

[P1] 9. Enforce Security Utilities

Import and use `sanitizeInput` from `security.ts` in all API routes that accept user input. Add CSRF token generation and validation to all mutation endpoints. Move rate limiting to a database-backed store (using Prisma) or use Vercel Edge Middleware. Validate `JWT_SECRET` at startup with a hard throw instead of a warning.

Files: `src/lib/security.ts`, all route files, `src/lib/rate-limit.ts`

[P2] 10. Add Accessibility Foundations

Add `@media (prefers-reduced-motion: reduce)` to `globals.css` to disable all animations for users with motion sensitivity. Add `aria-current="page"` to active navigation items. Add a skip-to-content link before the sidebar. Change password toggle `tabIndex` from -1 to 0. Add `role="alert"` to error messages in the login view. These are low-effort, high-impact accessibility improvements.

Files: `globals.css`, `sidebar.tsx`, `login-view.tsx`, `page.tsx`

16. What Should NOT Be Changed

Despite the extensive issues identified, several aspects of ELASTICO represent genuine engineering strength and should be preserved during any remediation work.

16.1 Mathematical Prediction Engine

The stochastic prediction engine (`src/lib/prediction-engine.ts`) is a genuine technical asset. The implementation of Merton Jump-Diffusion, GARCH(1,1), Bivariate Correlated Poisson, and Monte Carlo simulation with Kelly Criterion is sophisticated and correct. The 20 proprietary formulas in `src/lib/advanced-analytics-engine.ts` (Psychological, Temporal, Tactical, Market, Abyss tiers) represent meaningful intellectual property. These should not be simplified, replaced, or removed.

16.2 AI Gateway Architecture

The 7-provider failover chain in `src/lib/ai-gateway.ts` is well-designed. The cooldown system, input limits, streaming support, and automatic failover represent a robust approach to AI provider reliability. The transparent mock-fallback with metadata flagging is an honest design pattern. The architecture is sound and should be preserved; it needs conversation history and source attribution added, not replaced.

16.3 ESPN Data Pipeline

The ESPN public API integration provides genuine real-time data for 20 leagues without requiring an API key. The 12 action types and the `/api/sync` upsert pattern are reliable and well-implemented. This is the backbone of the application and should be protected.

16.4 Authentication System

JWT with bcrypt hashing, account lockout, IP rate limiting, and session restoration represent a complete authentication implementation. The security issues identified (demo login, in-memory rate limiting) are fixable without replacing the core auth architecture.

16.5 Database Schema

The 15-model Prisma schema covers the core domain well: User, Session, Team, Player, Match, MatchEvent, Prediction, Vote, Bookmark, NewsItem, Activity, ApiLog, SystemSetting, Announcement, FeatureFlag, Notification. The relationships are appropriate and the schema supports the core workflows.

16.6 Component Library (shadcn/ui)

The 41 shadcn/ui components provide a solid, consistent component foundation. These are standard Radix UI primitives with Tailwind styling. The component library is not the problem; the problem is how the application uses (or fails to use) them.

17. Implementation Roadmap

The following phased roadmap orders fixes by dependency and impact. Each phase should be completed and verified before starting the next. Estimated effort is relative, not absolute.

Phase 0: Critical Functional Fixes

- Restrict demo login endpoint (remove auto-create admin capability)
- Add loading states to all views (Skeleton components from shadcn/ui)
- Remove or clearly label all fake data with "DEMO DATA" badges
- Fix dead UI elements (forgot password, terms links, zoom controls)
- Fix duplicate polling in page.tsx and SetupView

Phase 1: Information Architecture

- Restructure sidebar navigation (group analytical tools, reduce to 7-9 items)
- Implement URL-based routing (convert Zustand switching to App Router pages)
- Add code splitting via React.lazy or Next.js dynamic imports
- Fix missing view titles in header (add 7 missing entries to viewTitleMap)
- Add breadcrumb or view indicator for user orientation

Phase 2: Core Workflows

- Add AI conversation history (Prisma Conversation model, context in API)
- Fix dashboard information hierarchy (remove duplicates, empty placeholders)
- Make model selector functional (pass model tier to API)
- Fix bookmark toggle to call /api/bookmarks API
- Add empty states with contextual guidance to all views

Phase 3: Data Integrity

- Build real API endpoints for admin dashboard metrics
- Build real API endpoints for tactical analysis data
- Build real API endpoints for social features (or remove social view)
- Build real API endpoints for compare view H2H data
- Create unified league configuration (single source of truth)

Phase 4: Analytics Quality

- Fix player radar charts to use real data from API
- Fix compare view to show team-specific data
- Add chart empty/loading/error states
- Fix dashboard chart labeling (xgChartData naming)
- Fix system monitor to use API responses instead of Math.random()

Phase 5: AI Experience

- Implement conversation history in /api/chat
- Add source attribution to AI responses (which data informed the analysis)
- Display visual indicator when AI falls back to mock template
- Add AI confidence scoring and uncertainty communication
- Make suggested prompts context-aware (based on current match/view)

Phase 6: Security Hardening

- Enforce security.ts utilities across all API routes
- Move rate limiting to database-backed store
- Add CSRF protection to mutation endpoints
- Make JWT_SECRET validation hard-fail at startup
- Add input validation (zod) to all API routes

Phase 7: Performance

- Remove unused npm dependencies (8+ packages)
- Migrate from manual fetch to @tanstack/react-query
- Add Next.js Image optimization for team logos and player photos
- Implement request prioritization for initial data fetches
- Add service worker back for offline caching (PWA re-enable)

Phase 8: Accessibility

- Add prefers-reduced-motion media query to globals.css
- Add aria-current="page" to active navigation items
- Add skip-to-content link
- Fix password toggle keyboard accessibility (tabIndex)
- Add role="alert" to error messages
- Add non-Latin font subsets (CJK, Arabic) for player names

Phase 9: Visual Polish

- Remove ~200 lines of unused CSS from globals.css
- Remove duplicate .dark block and duplicate color definitions
- Fix icon collisions (Target, Shield) in sidebar
- Add light theme support or remove ThemeProvider overhead
- Consistent border radius values across components
- Responsive mobile optimization for tables, charts, and navigation